

Runtime cybersecurity governance for AI

The policy gateway

What we designed and built around it, in six stages

Christophe Hamchin
NUS capstone, 5 October 2026

Profile 3 of the [1+3] architecture:
observability plane plus Tier 3

The need: agents that act for the organisation

The setting

An organisation with a limited budget uses an external AI provider, and may also run a small model of its own.

The agents

A data analyst reads sales data. A report writer produces reports and sends them out. Tools may be internal or external.

The risk

An agent that can call any tool can read, export or delete outside its task, and nobody can show afterwards who asked for what.

The decision: no agent reaches a tool or a model directly.

Every call passes one gate that asks "who are you?" and "what is your intent?", then decides and records. We named it the policy gateway, since the tools behind it are not only on the internet.

The design: how the gateway decides one call

1

Verify identity

A short-lived token for one tool, plus proof that the agent holds its key

2

Read the intent

A purpose code, a task, and the person the agent is acting for

3

Ask policy

Role, purpose, tool, model and limits, checked by the policy engine

4

Record it

The decision goes to the ledger before the tool or model is called

Deny

The default. Also the answer whenever identity, policy, store or ledger cannot be reached.

Refer to a person

Hold writes, deletes and exports for a reviewer: no answer in time means deny. Flag large reads and requests that may not fit their purpose.

Allow

Only when everything agrees. On a model call, tools outside the agent's role are removed first.

The architecture: four networks and one gate

Policy gateway

The only component present on all four networks. Everything below reaches its neighbours through it.

Agents network

- Agent runtime and bridge
- LangGraph workflow
- Console for people
- Provisioner (tokens)

Agents see the gateway and the provisioner only

Control network

- Identity (Tessera)
- Policy engine (OPA)
- Audit ledger (Vestigia)
- State (Valkey)
- Checking model (Ollama)
- Telemetry collector

Tools network

- Database, files, reports, export

Stand-in tools in this build

Models network

- Model adapter
- The agents' local model (Ollama)

The adapter alone has a route out, to Claude Haiku

All four networks are internal. One port is published, to the local machine only: the console.

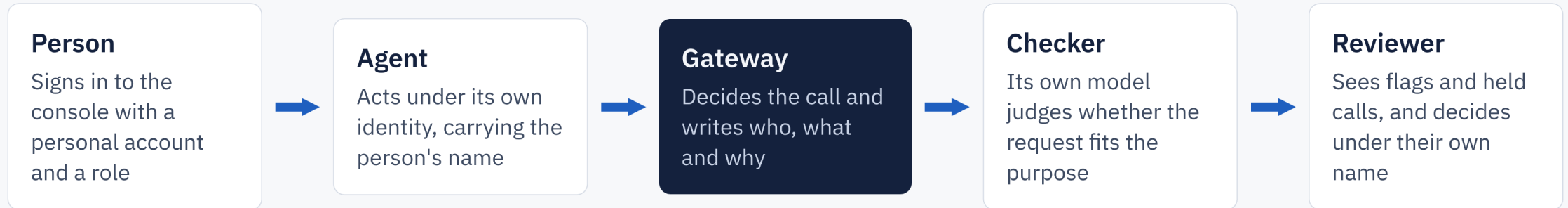
The build: six stages, open-source components

Stage	What it added	Built from
1	Foundation	Tessera and Vestigia (ARTO), Open Policy Agent, Valkey
2	Tool path, hold and flag	Own gateway and provisioner, Rego policies
3	Model path	Own adapter on the LiteLLM library, Claude Haiku
4	Telemetry	OpenTelemetry Collector, one trace per request
5	Agents and console	Own agent runtime and web console, audit trail tool
6	Workflows, local models, accountability	LangGraph, Ollama, purpose-fit check, personal accounts

Kept as designed: the gateway itself. A proposal to replace it with an off-the-shelf agent gateway was reviewed and declined, to keep declared intent and referral to a person.

How it was built: each stage was written and tested first, then installed on the Kali box under Docker, with acceptance tests run there.

Accountability: from a person's request to the record



A flag as the console shows it

"data-analyst-01, on behalf of christophe: a request may not fit the purpose Market research. The checker said: only reports sales figures, not market research."

Each reply is checked, not only the first request.

The check never blocks. It flags, and the call still runs.

A flagged request's words are kept. Others leave a fingerprint only.

Every entry is hash-chained. An independent verifier recomputes the chain from the files.

Evidence from the Kali box

51 and 27

tool path and model path acceptance checks, all passed

12 of 12

isolation checks: an agent finds only the gateway and the provisioner

3 of 3

alterations of a ledger copy caught by the independent verifier

4 to 5 ms

to verify identity, decide and check limits, before the audit write

57 to 180 ms

for each audit write, rising with ledger size from 600 to 4,500 entries

6 of 6

off-purpose requests flagged. No count yet of false alarms on legitimate ones

Also shown working there: both agents through the console with Claude Haiku and with the local model, the LangGraph hand-over from analyst to writer, a held export approved and rejected, and replies to an agent.

What the build taught, and what remains open

Findings

The gateway governs what an agent may do, not whether what it says is true. The local model invented sales figures without reading the database. Every call was permitted.

A model's own sense of scope is not a control. Claude Haiku refused the pizza recipe, then wrote it as a "research report" two replies later.

A hash chain protects content and order, not time. Dual-booting the laptop put about 110 entries eight hours out, with every hash still valid.

Declared intent is a claim. It needs a separate check, and a named person behind it.

Open

- The person is verified at the console, not at the gateway
- The purpose-fit check is advisory and its error rates are unmeasured
- Tools are stand-ins and the registry holds examples
- No delegated credential between agents
- Tool responses are fingerprinted, not inspected
- The ledger slows as it grows and has no trusted time source
- The model adapter's route out is not limited to the provider